A Appendices for "Improving the Performance of Spike-Based Deep Q-Learning Using Ternary Neurons"

A.1 Limitation

In this work, we address key challenges of spiking neural networks (SNNs) in Q-learning by enhancing their representational capacity through a novel asymmetric ternary spiking neuron model. However, Q-learning is inherently limited to environments with discrete action spaces and cannot directly handle tasks with continuous action spaces. For such problems, including many control applications, alternative reinforcement learning approaches, such as actor-critic methods, are more appropriate. Extending our framework to these settings is a direction for future research. Additionally, our focus in this work has been primarily on algorithmic development and theoretical validation, with the goal of establishing a reliable and energy-efficient framework for onboard autonomous decision making. We do not address the hardware implementation or deployment aspects of the proposed models, which remain to be explored in future studies.

A.2 Broader Impacts

This work introduces a foundational mathematical framework for analyzing the representation and learning dynamics of SNNs, which paves the way for a more principled design and analysis of neuromorphic models. Using this framework, we propose a novel ternary spiking neuron model that significantly enhances the representational capacity of SNNs. Although demonstrated in the context of Q-learning, the proposed model and framework are broadly applicable and hold promise for a wide range of machine learning domains, including computer vision, natural language processing, and control systems. By improving the representation capacity of SNNs, which is a widely recognized challenge for developing applications of neuromorphic computing, this research contributes to closing the gap between energy-efficient neuromorphic systems and traditional deep learning models. These advances may accelerate the deployment of SNN-based solutions in low-power settings such as edge computing, autonomous robotics, and embedded AI. Moreover, the theoretical insights provided here may inform future innovations in neuromorphic computing and spur interdisciplinary research at the intersection of neuroscience, machine learning, and hardware design.

A.3 LIF Neuron Dynamics

We analyze the dynamical behavior of LIF neurons. Consider an LIF neuron located in the first hidden layer of a large SNN. This neuron receives input in the form of spikes from the preceding (input) layer, each modulated by its corresponding synaptic weight. In the subthreshold regime, where the membrane potential remains below the firing threshold, the neuron's dynamics can be described by the continuous-time version of the LIF model (i.e., the continuous analog of Equation 1 of the main paper) as

$$\tau \frac{dm(t)}{dt} = -\left(m(t) - V_{\text{reset}}\right) + \sum_k \sum_{t_k^i} w_k \, \delta(t - t_k^i), \tag{9}$$

where τ is the membrane time constant; w_k denotes the synaptic weight between the target neuron and the k-th input; t_k^i represents the time of the i-th spike from the k-th input; $\delta(t)$ is the Dirac delta function; and $m(t)$ and V_{reset} denote the membrane potential and the reset potential, respectively [1].

Assuming rate-based encoding in the input layer, where raw input values are converted into spike trains via Bernoulli sampling, and that the inputs are independent or only weakly correlated (a condition often satisfied in practice), the Central Limit Theorem implies that the aggregated input to a neuron can be approximated by a Gaussian distribution. To illustrate this in our setting, consider the first convolutional layer of the network, which uses an 8×8 kernel and operates on inputs with 4 channels. Thus, we consider a flattened input of size 256, with each element independently drawn from a uniform distribution over $[0, 1]$, mimicking normalized raw pixel intensities. Each input element produces a spike with a probability proportional to its magnitude. For synaptic weights w_k, we use the kernel parameters of one of the best-performing models.

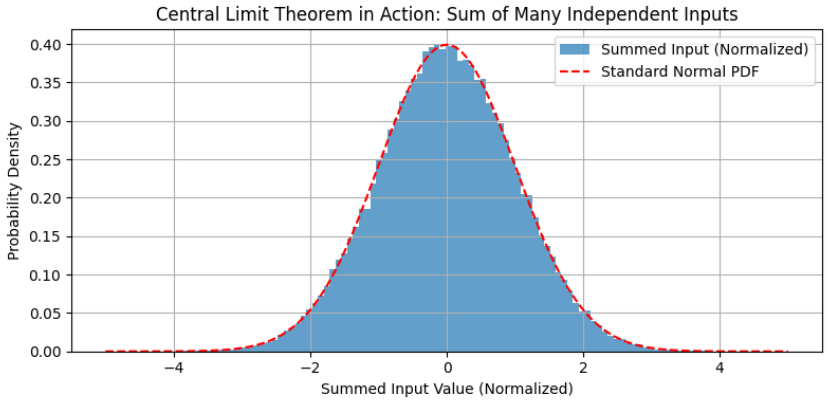

Figure 3: Approximate Gaussian distribution of the input term $\sum_k \sum_{t_k^i} w_k\, \delta(t - t_k^i)$, with 256 uniformly distributed inputs converted to spike trains via Bernoulli sampling.

Figure 3 shows the resulting distribution of the term $\sum_k \sum_{t_k^i} w_k\, \delta(t - t_k^i)$, which represents the total weighted input current to a postsynaptic neuron. Thus, Equation 9 can be written as follows:

$$\tau \frac{dm(t)}{dt} = -\left(m(t) - V_{\text{reset}}\right) + \rho(t), \tag{10}$$

where $\rho(t)$ is a white noise process representing the aggregated input $\sum_k \sum_{t_k^i} w_k\, \delta(t - t_k^i)$. We note that Equation 10 describes the *Ornstein–Uhlenbeck stochastic process* which satisfies the *Fokker–Planck equation* governing the evolution of the probability density function associated with the membrane potential dynamics [1]. This equation is given by

$$\tau \frac{\partial}{\partial t} p(m,t) = -\frac{\partial}{\partial m}\left[\left(-m + V_{\text{reset}} + \tau \sum_k w_k \nu_k(t)\right) p(m,t)\right] + \frac{1}{2}\xi^2 \frac{\partial^2}{\partial m^2} p(m,t), \tag{11}$$

where $p(m,t)$ denotes the probability density of the neuron having membrane potential m at time t, and $\nu_k(t)$ represents the spike rate of the k-th input. The term ξ is the time-dependent noise amplitude, defined as follows:

$$\xi^2 = \tau \sum_k w_k\, \nu_k^2(t).$$

In the subthreshold regime, assuming $\mathbb{E}[\rho(t)] \approx 0$, the expected trajectory of the membrane potential can be obtained by solving Equation 10, yielding the expression

$$m_0(t) = \mathbb{E}[m(t)] = V_{\text{reset}}\left(1 - e^{-t/\tau}\right).$$

Fluctuations in the membrane potential have a variance σ_m^2, which can be calculated as

$$\sigma_m^2(t) = \frac{1}{2}\xi^2\left(1 - e^{-t/\tau}\right).$$

Finally, the solution to Equation 11 can be calculated as [1]

$$p(m,t) = \frac{1}{\sqrt{2\pi\sigma_m^2(t)}} \exp\left(-\frac{(m(t) - m_0(t))^2}{2\sigma_m^2(t)}\right). \tag{12}$$

Equation 12 describes that $p(m,t)$ follows a Gaussian-like distribution centered around $m_0(t)$. In most practical applications, it is common to assume $V_{\text{reset}} = 0$, in which case $p(m,t)$ is centered around zero (note to $m_0(t)$). Although Equation 12 describes the subthreshold behavior of an LIF neuron without a firing threshold, it can also be used to approximate the distribution of $m(t)$ in LIF neurons with a spike-based threshold. In such cases, the spike threshold acts as an absorbing barrier: when the membrane potential reaches this threshold, it triggers a spike and resets, effectively truncating the upper tail of the distribution [1]. Combined with the reset mechanism, this causes the membrane potential to spend more time near V_{reset}, resulting in a distribution that is typically a truncated version of a Gaussian.

Table 2: Commonly used surrogate gradient functions in SNNs

Name	Surrogate Function	Surrogate Gradient
Atan	$s^l(t) = \frac{1}{\pi}\arctan\left(\frac{\pi m(t)\alpha}{2}\right)$	$GE = \frac{\partial s^l(t)}{\partial m(t)} = \frac{1}{\pi}\cdot\frac{1}{1+\left(\frac{\pi m(t)\alpha}{2}\right)^2}$
STE	$s^l(t) = m(t)$	$GE = \frac{\partial s^l(t)}{\partial m(t)} = 1$
Sigmoid	$s^l(t) = \frac{1}{1+\exp(-km(t))}$	$GE = \frac{\partial s^l(t)}{\partial m(t)} = \frac{k\exp(-km(t))}{(\exp(-km(t))+1)^2}.$

A.4 Expected Value of Gradient and Spike Derivatives

In this section, we calculate the expected value of the gradient estimator and the expected gradient of the stochastic spike output, as defined in Equations 3 and 4 of the main paper.

Commonly Used Gradient Estimators

As discussed in Section 1 of the main paper, the non-differentiable nature of the threshold function poses a challenge for gradient-based learning. Surrogate Gradient Learning (SGL) addresses this issue during backpropagation by replacing the true threshold function with a differentiable pseudo-function. This surrogate allows the gradient to be estimated using the derivative of the pseudo-function. Some commonly used surrogate functions are listed in Table 2 [2]. Here, α and k are model parameters, usually set to 2 and 25, respectively.

The expected value of the gradient estimators (surrogate gradient functions) can be computed as

$$\mathbb{E}[GE(m,t)] = \int GE(m)\, p(m,t)\, dm. \tag{13}$$

Based on the commonly used pseudo-functions listed in Table 2 and the form of $p(m,t)$ given in Equation 12, both $GE(m)$ and $p(m,t)$ are strictly positive for all m. Consequently, as shown in Equation 13, the expected value $\mathbb{E}[GE(m,t)]$ is strictly greater than zero.

Stochastic Spike Gradient

Equations 6 and 7 (in the main paper) describe the expected value of the spike-generation gradient for binary and ternary neurons, respectively. The first step in computing these values is to evaluate the probabilities p^+, p^0, and p^- defined in Equations 3 and 4 (in the main paper). Using the results of Section A.3, these probabilities can be calculated for binary spiking neurons as

$$p^+ = \int_{v^{\text{th}}}^{\infty} p(m,t)\, dm, \quad p^0 = \int_{-\infty}^{v^{\text{th}}} p(m,t)\, dm \tag{14}$$

and for ternary spiking neurons as

$$p^+ = \int_{v^{\text{th}}}^{\infty} p(m,t)\, dm, \quad p^0 = \int_{-v^{\text{th}}}^{v^{\text{th}}} p(m,t)\, dm, \quad p^- = \int_{-\infty}^{-v^{\text{th}}} p(m,t)\, dm \tag{15}$$

We make the following key observations.

- For *ternary neurons with symmetric thresholds*, where $v^{\text{th}_n} = v^{\text{th}_p}$, we have $p^+ = p^-$. (according to Equations 12 and 15). Thus, based on Equation 7 (from the main paper), regardless of $m(t)$, the expected value of the gradient becomes zero.
- For *ternary neurons with asymmetric thresholds*, where $v_p^{\text{th}} \neq v_n^{\text{th}}$, we have $p^+ \neq p^-$ which results in a non-zero expected gradient.
- For *binary neurons*, using Equations 6 (from the main paper), 12, and 14, the expected value of the gradient is also non-zero.

A.5 Proofs of Lemma and Theorem in Section 4.3

Lemma 4.6. *If the spike firing rate of neurons is r, and the network uses asymmetric ternary spiking neurons as described in Equation 8, then $\phi(J_j J_j^T) = 1 - r$ and $\varphi(J_j J_j^T) = r - r^2$.*

Proof. In networks with asymmetric ternary spiking neurons, the key factor in computing the Jacobian is the gradient of the threshold function defined in Equation 8 (from the main paper). We denote its Jacobian as s_m, which forms a diagonal matrix due to the element-wise nature of the function. During backpropagation, gradient estimators such as STE are applied. Given the reset mechanism in Equation 1 (from the main paper), the membrane potential $m(t)$ remains within $(-v^{th_n}, v^{th_p})$, so the entries of s_m are 1 when $m(t) \in (-v^{th_n}, v^{th_p})$, and 0 otherwise. Thus, the probability density function of s_m can be defined as:$\rho_{s_m}(z) = r\delta(z) + (r-1)\delta(z-1)$. Since the values of s_m matrix are binary, it follows that $\rho_{s_m s_m^T}(z) = \rho_{s_m}(z)$. Thus,

$$\phi(s_m s_m^T) = \int z\, \rho_{s_m s_m^T}(z)\, dz = 1 - r \tag{16}$$

$$\varphi(s_m s_m^T) = \int z^2\, \rho_{s_m s_m^T}(z)\, dz - \varphi^2(s_m s_m^T) = r - r^2$$

□

Theorem 4.7. *The asymmetric ternary spiking neuron achieves at least the dynamical isometry that of ReLU activations, that is, $\phi(s_m s_m^T) \geq \phi(J_j J_j^T)$ and $\varphi(s_m s_m^T) \leq \varphi(J_j J_j^T)$.*

Proof. The value of p for the zero-mean input distribution is around 0.5, while the firing rate in SNNs is usually between 0.1 and 0.5. Thus, based on Lemmas 4.5 and 4.6, we conclude that $\phi(s_m s_m^T) \geq \phi(J_j J_j^T)$ and $\varphi(s_m s_m^T) \leq \varphi(J_j J_j^T)$. □

A.6 Experimental Details

Experimental Setup

We conducted our experiments using the Atari environments from the OpenAI Gym environment. Each environment provides the agent with a stack of four consecutive grayscale frames, which serve as input for the agent to select an action from the available discrete action set. We ran each experiment using five different random seeds. We adopt the discount factor, target network update frequency, and batch size values from the settings used in [3]. To prevent out-of-memory issues, we set the replay buffer start size to $10{,}000$ and the total buffer capacity to $400{,}000$. The hardware specifications of the machine used to perform the experiments are listed in Table 3.

Network Architecture and Hyperparameters We used a DQN consisting of three convolutional layers followed by two fully connected layers. The architecture parameters of the convolutional layers are summarized in Table 4. All neurons in the network are spiking neurons of the same type, depending on the selected variant, DSQN, DTSQN, or DATSQN. For gradient estimation during backpropagation, we used the arctangent (Atan) function as a surrogate gradient. While we also tested Sigmoid and STE as alternatives, both led to degraded performance across all model variants.

The hyperparameters of the spiking neurons were selected as follows: We set the positive threshold voltage to $v^{\text{th}_p} = v^{\text{th}} = 1$, following the empirical findings reported in [4, 5], which demonstrated performance improvements with this choice. The negative threshold v^{th_n} was initialized to 2 and implemented as a learnable parameter using `nn.Parameter()`, allowing it to be updated through standard gradient-based optimization along with other network weights. The optimal values for the decay factor β and the learning rate were selected by trial and error using the Breakout game, as these parameters significantly affect both the training dynamics and the final performance. The complete list of the hyperparameters used in the SNNs is provided in Table 5. The process of forward propagation through the network is presented in Algorithm 1.

Input Encoding For input encoding, we first normalized the input pixel values to the range $[0, 1]$ by dividing them by 255. Spikes were then generated using Bernoulli sampling, where the probability of

Table 3: Hardware specification.

Component	Specification
CPU	Intel(R) Core(TM) i7-14700k
GPU	NVIDIA GeForce RTX 4060
OS	Ubuntu 24.04.1 LTS
Memory	16 GB

Table 4: Parameters of the convolution layers.

Layers	Num. kernels	Kernel size	Stride
conv1	32	8×8	4
conv2	64	4×4	2
conv3	64	3×3	1

Table 5: Parameters of the spiking neuron model.

Parameter	Value
Simulation time window ($\mathcal{T}$)	20
Decay factor (β)	0.9
Firing threshold (v^{th} or v^{th_p})	1
Reset potential (v_{reset})	0
Learning rate	0.00005

Algorithm 1 Forward propagation through DSQN-DTSQN-DATSQN

Randomly initialize weight and kernel matrices $\mathbf{W}$ and biases $\mathbf{b}$ for each SNN layer;
Initialize membrane potentials, $v^k(0)$ by 0;
$(M \times N \times N)$-dimensional observation state, I;
Spikes from input generated by the rate encoder module: $\mathbf{X} = \text{Encoder}(\mathbf{I})$;
for $t = 1, \ldots, T$ **do**
    Spikes from input encoder at timestep t: $s^0(t) = \mathbf{X}(t)$
    for $k = 1, \ldots, K-1$ **do**
        Update LIF neurons in layer k at timestep t based on spikes from layer $k-1$:
        $m^k(t) = v^k(t-1) + \mathbf{W}^k s^{k-1}(t) + \mathbf{b}^k$
        $s^k(t) = \text{Threshold}(m^k(t)$
        $v^k(t) = \beta m^k(t)(1 - s^k(t)) + v_{\text{reset}} s^k(t)$
Sum up the spikes of output layer: $\mathbf{Q} = \sum_{t=1}^{T} \mathbf{W}^K s^{K-1}(t) + \mathbf{b}^K$

Table 6: Performance difference between DTSQN & DATSQN.

Game	DTSQN score (%)	DATSQN score (%)
Beam Rider	73%	193%
Boxing	62%	100%
Breakout	11%	121%
Crazy Climber	8%	102%
Gopher	63%	114%
Jamesbond	65%	135%
Space Invaders	71%	143%
Average (%) w.r.t. DSQN	50%	130%

firing was proportional to the normalized pixel intensity. We also experimented with the rate encoding module from the *snnTorch* library [2] , which generates spikes based on a Poisson distribution. The results remained largely consistent across both methods. This input encoding approach was applied across all model variants: DSQN, DTSQN, and DATSQN.

A.7 Quantitative Comparison Between DTSQN and DATSQN

To highlight the performance gains of the proposed asymmetric ternary neuron model and the degradation observed with the state-of-the-art ternary model in Q-learning tasks, we normalize the scores of both DTSQN and DATSQN relative to the baseline binary neuron-based DSQN. The results of this comparative analysis are presented in Table 6.

References

[1] Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.

[2] Jason K Eshraghian, Max Ward, Emre O Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.

[3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[4] Yufei Guo, Yuanpei Chen, Xiaode Liu, Weihang Peng, Yuhan Zhang, Xuhui Huang, and Zhe Ma. Ternary spike: Learning ternary spikes for spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12244–12252, 2024.

[5] Yulong Huang, Xiaopeng Lin, Hongwei Ren, Haotian Fu, Yue Zhou, Zunchang Liu, Biao Pan, and Bojun Cheng. Clif: Complementary leaky integrate-and-fire neuron for spiking neural networks. *arXiv preprint arXiv:2402.04663*, 2024.